Technical Documentation

ChequePrint Pro — System Architecture & Developer Reference

Version: 1.0.0
Platform: Electron.js (Windows / macOS / Linux)
Last Updated: April 2026

1. System Architecture

1.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                        Electron Shell                        │
│  ┌──────────────────────┐   ┌──────────────────────────┐    │
│  │    Renderer Process  │   │       Main Process       │    │
│  │  (React + Tailwind)  │◄──►  (Node.js + SQLite)      │    │
│  │                      │   │                          │    │
│  │  - UI Components     │   │  - Print Engine          │    │
│  │  - Zustand Store     │   │  - File System Access    │    │
│  │  - Form Validation   │   │  - Database Management   │    │
│  │  - Preview Canvas    │   │  - IPC Handlers          │    │
│  └──────────────────────┘   └──────────────────────────┘    │
│                  IPC Bridge (preload.js)                     │
└──────────────────────────────────────────────────────────────┘
                              │
                    ┌─────────▼──────────┐
                    │   OS Print API     │
                    │   SQLite Database  │
                    │   File System      │
                    └────────────────────┘
```

1.2 IPC Communication Model

All communication between renderer and main process uses Electron's `contextBridge` with strict channel whitelisting.

Allowed IPC channels:

Channel	Direction	Purpose
`print:cheque`	Renderer → Main	Trigger single cheque print
`print:batch`	Renderer → Main	Trigger batch print job
`print:test`	Renderer → Main	Print calibration test page

Channel	Direction	Purpose
`db:save`	Renderer → Main	Save cheque record to SQLite
`db:query`	Renderer → Main	Query cheque records
`db:export`	Renderer → Main	Export records as CSV
`settings:get`	Renderer → Main	Read application settings
`settings:set`	Renderer → Main	Write application settings
`dialog:openFile`	Renderer → Main	Open file picker for CSV import

2. Database Schema

2.1 SQLite Tables

```
-- Bank templates (seeded from bankTemplates.js on first run)
CREATE TABLE IF NOT EXISTS bank_templates (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  bank_name   TEXT NOT NULL,
  bank_code   TEXT UNIQUE NOT NULL,
  width_mm    REAL NOT NULL,
  height_mm   REAL NOT NULL,
  orientation TEXT DEFAULT 'landscape' CHECK(orientation IN ('portrait','landscape'))
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
  is_custom   INTEGER DEFAULT 0
);

-- Fields within each bank template
CREATE TABLE IF NOT EXISTS template_fields (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  template_id INTEGER NOT NULL REFERENCES bank_templates(id) ON DELETE CASCADE,
  field_name  TEXT NOT NULL,   -- 'payee', 'amount_num', 'amount_words', 'date'
  x_mm        REAL NOT NULL,   -- X position from left edge
  y_mm        REAL NOT NULL,   -- Y position from top edge
  max_width_mm REAL,           -- Max printable width for this field
  font_size   INTEGER DEFAULT 10,
  font_face   TEXT DEFAULT 'Arial',
  is_bold     INTEGER DEFAULT 0
);

-- Printer calibration offsets per bank template
CREATE TABLE IF NOT EXISTS printer_calibration (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  template_id INTEGER NOT NULL REFERENCES bank_templates(id) ON DELETE CASCADE,
  printer_name TEXT NOT NULL,
  offset_x_mm REAL DEFAULT 0,
  offset_y_mm REAL DEFAULT 0,
  updated_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Cheque records (printed history)
```

```
CREATE TABLE IF NOT EXISTS cheque_records (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  cheque_number TEXT,
  bank_code     TEXT NOT NULL,
  payee_name    TEXT NOT NULL,
  amount        REAL NOT NULL,
  amount_words  TEXT NOT NULL,
  cheque_date   TEXT NOT NULL,  -- DD/MM/YYYY
  account_ref   TEXT,
  ifsc_code     TEXT,
  narration     TEXT,
  status        TEXT DEFAULT 'printed' CHECK(status IN ('printed','pending','cancelle
  cancel_reason TEXT,
  printed_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
  batch_id      TEXT            -- UUID for batch-grouped cheques
);

-- Application settings (key-value store)
CREATE TABLE IF NOT EXISTS app_settings (
  key   TEXT PRIMARY KEY,
  value TEXT NOT NULL
);
```

2.2 Default Settings Seeds

```
INSERT OR IGNORE INTO app_settings VALUES ('default_bank', 'SBI');
INSERT OR IGNORE INTO app_settings VALUES ('auto_increment_cheque', '1');
INSERT OR IGNORE INTO app_settings VALUES ('last_cheque_number', '000001');
INSERT OR IGNORE INTO app_settings VALUES ('date_format', 'DD/MM/YYYY');
INSERT OR IGNORE INTO app_settings VALUES ('currency_symbol', '₹');
INSERT OR IGNORE INTO app_settings VALUES ('default_font', 'Arial');
INSERT OR IGNORE INTO app_settings VALUES ('default_font_size', '10');
INSERT OR IGNORE INTO app_settings VALUES ('backup_path', '');
```

3. Bank Template Data Structure

3.1 Template Schema (JavaScript)

```
// src/data/bankTemplates.js
const bankTemplates = [
  {
    bank_code: "SBI",
    bank_name: "State Bank of India",
    width_mm: 176,
    height_mm: 84,
    orientation: "landscape",
    fields: {
      payee: { x: 34, y: 22, maxWidth: 120, fontSize: 11, bold: false },
      amount_num: { x: 130, y: 22, maxWidth: 40, fontSize: 11, bold: false },
      amount_words: { x: 10, y: 38, maxWidth: 155, fontSize: 10, bold: false },
      date_dd: { x: 136, y: 8, maxWidth: 8, fontSize: 11, bold: false },
```

```
      date_mm: { x: 148, y: 8, maxWidth: 8, fontSize: 11, bold: false },
      date_yyyy: { x: 158, y: 8, maxWidth: 16, fontSize: 11, bold: false }
    }
  },
  // ... additional banks
];
```

> **Note:** All position values are in millimetres (mm) from the top-left corner of the cheque leaf. Measurements must be verified against actual cheque leaves using a ruler before production use.

4. Print Engine

4.1 Print Flow

```
User clicks [Print]
    |
    ▼
Validate cheque fields (Zod schema)
    |
    ▼
Fetch bank template + calibration offset from SQLite
    |
    ▼
Build print payload:
  { fields, positions, paperSize, orientation, dpi, offsets }
    |
    ▼
IPC: renderer → main (channel: print:cheque)
    |
    ▼
Main process: generate HTML print document
    |
    ▼
electron-print: send to OS print API with custom paper size
    |
    ▼
Save record to SQLite (status: 'printed')
    |
    ▼
Show success toast in renderer
```

4.2 HTML Print Document Generation

The print engine generates a minimal HTML document for each cheque. CSS `@page` rules are used to set the exact paper dimensions:

```
// electron/print.js
function buildPrintHTML(chequeData, template, offsets) {
  const { width_mm, height_mm } = template;
```

```
  const fields = chequeData;

  return `
<!DOCTYPE html>
<html>
<head>
<style>
  @page {
    size: ${width_mm}mm ${height_mm}mm;
    margin: 0;
  }
  body {
    margin: 0;
    padding: 0;
    width: ${width_mm}mm;
    height: ${height_mm}mm;
    position: relative;
    overflow: hidden;
  }
  .field {
    position: absolute;
    font-family: Arial, sans-serif;
    white-space: nowrap;
  }
</style>
</head>
<body>
  ${generateFieldHTML(fields, template, offsets)}
</body>
</html>`;
}

function generateFieldHTML(fields, template, offsets) {
  return Object.entries(template.fields).map(([key, pos]) => {
    const value = fields[key] || '';
    const x = pos.x + (offsets?.x || 0);
    const y = pos.y + (offsets?.y || 0);
    return `<div class="field" style="
      left: ${x}mm;
      top: ${y}mm;
      font-size: ${pos.fontSize}pt;
      font-weight: ${pos.bold ? 'bold' : 'normal'};
      max-width: ${pos.maxWidth}mm;
    ">${escapeHTML(value)}</div>`;
  }).join('\n');
}
```

4.3 Custom Paper Size

On Windows, custom paper sizes are registered via the Win32 Print API using `DEVMODE` structures. On macOS, CUPS is used. The application handles this automatically via the `electron-printer` library.

5. Indian Number-to-Words Module

5.1 Algorithm

```
// src/lib/numberToWords.js

const ones = ['', 'One', 'Two', 'Three', 'Four', 'Five', 'Six', 'Seven',
              'Eight', 'Nine', 'Ten', 'Eleven', 'Twelve', 'Thirteen',
              'Fourteen', 'Fifteen', 'Sixteen', 'Seventeen', 'Eighteen', 'Nineteen'

const tens = ['', '', 'Twenty', 'Thirty', 'Forty', 'Fifty',
              'Sixty', 'Seventy', 'Eighty', 'Ninety'];

function convertHundreds(n) {
  if (n === 0) return '';
  if (n < 20) return ones[n];
  if (n < 100) return tens[Math.floor(n / 10)] + (n % 10 ? ' ' + ones[n % 10] : '');
  return ones[Math.floor(n / 100)] + ' Hundred' +
    (n % 100 ? ' And ' + convertHundreds(n % 100) : '');
}

export function numberToIndianWords(amount) {
  if (amount === 0) return 'Zero Only';

  const crore = Math.floor(amount / 10000000);
  const lakh  = Math.floor((amount % 10000000) / 100000);
  const thou  = Math.floor((amount % 100000) / 1000);
  const rem   = amount % 1000;

  let result = '';
  if (crore) result += convertHundreds(crore) + ' Crore ';
  if (lakh)  result += convertHundreds(lakh)  + ' Lakh ';
  if (thou)  result += convertHundreds(thou)  + ' Thousand ';
  if (rem)   result += convertHundreds(rem);

  return result.trim() + ' Only';
}
```

6. CSV/Excel Import Module

6.1 Expected CSV Format

```
payee_name,amount,date,cheque_number,narration
Rajesh Kumar,25000,15/04/2026,000101,Office Rent
Sharma Traders,150000,15/04/2026,000102,Material Payment
```

6.2 Import Flow

```
// src/lib/csvImport.js
import Papa from 'papaparse';
import * as XLSX from 'xlsx';

export async function importCSV(filePath) {
  const text = await window.electronAPI.readFile(filePath);
  const result = Papa.parse(text, { header: true, skipEmptyLines: true });
  return validateRows(result.data);
}

export async function importExcel(filePath) {
  const buffer = await window.electronAPI.readFileBinary(filePath);
  const workbook = XLSX.read(buffer, { type: 'buffer' });
  const sheet = workbook.Sheets[workbook.SheetNames[0]];
  const data = XLSX.utils.sheet_to_json(sheet);
  return validateRows(data);
}

function validateRows(rows) {
  return rows.map((row, i) => {
    const errors = [];
    if (!row.payee_name) errors.push('payee_name is required');
    if (!row.amount || isNaN(Number(row.amount))) errors.push('amount must be numer
    if (!row.date || !/^\d{2}\/\d{2}\/\d{4}$/.test(row.date))
      errors.push('date must be DD/MM/YYYY');
    return { ...row, rowIndex: i + 2, valid: errors.length === 0, errors };
  });
}
```

7. Zod Validation Schema

```
// src/lib/validation.js
import { z } from 'zod';

export const chequeSchema = z.object({
  payee_name: z
    .string()
    .min(2, 'Payee name too short')
    .max(80, 'Payee name too long')
    .regex(/^[a-zA-Z0-9 .,'&()-]+$/, 'Invalid characters in payee name'),

  amount: z
    .number()
    .positive('Amount must be greater than zero')
    .max(999999999, 'Amount exceeds maximum limit (₹99,99,99,999)'),

  cheque_date: z
    .string()
    .regex(/^\d{2}\/\d{2}\/\d{4}$/, 'Date must be in DD/MM/YYYY format')
    .refine(val => {
      const [dd, mm, yyyy] = val.split('/').map(Number);
```

```
      const chequeDate = new Date(yyyy, mm - 1, dd);
      const today = new Date();
      const threeMonthsAgo = new Date();
      threeMonthsAgo.setMonth(today.getMonth() - 3);
      return chequeDate >= threeMonthsAgo;
    }, 'Cheque is stale (older than 3 months)'),

  bank_code: z.string().min(2),
  account_ref: z.string().optional(),
  ifsc_code: z.string().regex(/^[A-Z]{4}0[A-Z0-9]{6}$/).optional().or(z.literal('')).
  narration: z.string().max(200).optional()
});
```

8. Electron Build Configuration

```
// electron-builder.config.js
module.exports = {
  appId: 'com.chequeprintpro.app',
  productName: 'ChequePrint Pro',
  directories: { output: 'dist' },
  files: ['build/**/*', 'electron/**/*', 'node_modules/**/*'],
  win: {
    target: [{ target: 'nsis', arch: ['x64', 'ia32'] }],
    icon: 'assets/icons/icon.ico'
  },
  mac: {
    target: [{ target: 'dmg', arch: ['x64', 'arm64'] }],
    icon: 'assets/icons/icon.icns',
    category: 'public.app-category.business'
  },
  linux: {
    target: ['AppImage', 'deb'],
    icon: 'assets/icons/icon.png',
    category: 'Office'
  },
  nsis: {
    oneClick: false,
    allowToChangeInstallationDirectory: true,
    installerIcon: 'assets/icons/icon.ico',
    installerHeaderIcon: 'assets/icons/icon.ico'
  }
};
```

9. Error Handling & Logging

- All errors are caught at the IPC boundary and returned as structured `{ success: false, error: string }` objects.
- Renderer shows user-friendly messages; technical details are logged to `app.log` in the user data directory.
- Print failures trigger a retry dialog.

- Database errors trigger a backup prompt.

10. Performance Targets

Operation	Target
Application cold start	< 3 seconds
Database query (records list)	< 100ms
Print job dispatch	< 2 seconds
CSV import (1,000 rows)	< 1 second
Preview render on field change	< 50ms